

ContentSphere

Infrastructure as Code Deployment and Automation using Terraform

Project Report

Student Name	Saad Momin
Batch Code	DR501
Project Type	AWS DevOps / Infrastructure as Code
Primary Tool	Terraform
Additional Tools	AWS CloudFormation, AWS CDK, AWS CLI, Git
AWS Region	ap-south-1 (Mumbai)
Environment	dev
Repository	ContentSphere-Terraform-Infrastructure

1. Introduction

ContentSphere is an AWS infrastructure automation project designed to demonstrate Infrastructure as Code (IaC) using Terraform. The project provisions a complete basic AWS environment consisting of a VPC, public subnet, Internet Gateway, route table, security group, EC2 web server, Elastic IP and S3 bucket. The infrastructure is defined as reusable Terraform configuration rather than being created manually through the AWS Console.

The project also demonstrates remote Terraform state management using Amazon S3, infrastructure drift detection and correction, and alternative infrastructure definitions using AWS CloudFormation and AWS CDK. Git and GitHub are used for version control and project delivery.

2. Objectives

- Learn and implement Infrastructure as Code using Terraform.
- Create reusable Terraform modules for AWS resources.
- Provision a VPC and public networking components.
- Deploy an Ubuntu EC2 instance running Apache HTTP Server.
- Create and manage an Amazon S3 bucket through Terraform.
- Configure remote Terraform state using an S3 backend.
- Demonstrate infrastructure drift detection and correction.
- Understand AWS CloudFormation and AWS CDK as alternative IaC approaches.
- Use Git and GitHub to version and publish the project.
- Validate the final infrastructure using Terraform validation, planning and outputs.

3. Technologies and Tools Used

Technology / Tool	Purpose
Terraform	Primary Infrastructure as Code tool
AWS EC2	Compute instance hosting the Apache web server
Amazon VPC	Network isolation and IP address management
Amazon S3	Application bucket and remote Terraform state
AWS IAM / AWS account	Permissions and AWS resource access
AWS CloudFormation	Alternative declarative IaC implementation
AWS CDK	Infrastructure definition using TypeScript
AWS CLI	AWS resource validation and management
Git / GitHub	Source control and project hosting
Apache2	Web server installed on EC2

4. AWS Architecture

The Terraform configuration creates a simple public AWS architecture. A custom VPC uses the 10.0.0.0/16 CIDR range. A public subnet uses 10.0.1.0/24 and is associated with an Internet Gateway through a public route table. An Ubuntu EC2 instance is launched in the public subnet. The security group permits SSH on port 22 and HTTP on port

80. An Elastic IP provides a stable public address. An S3 bucket is created separately for application storage.

Architecture flow:

Internet → Internet Gateway → Public Route Table → Public Subnet → EC2/Apache Web Server

Terraform → AWS Provider → VPC / Subnet / IGW / Route Table / Security Group / EC2 / EIP / S3

5. Terraform Project Structure

The project is organized into a root Terraform configuration and reusable modules. The following structure was used:

```
ContentSphere-Terraform-Infrastructure/
├── README.md
├── provider.tf
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars.example
├── .gitignore
├── modules/
│   ├── vpc/
│   ├── security-group/
│   ├── ec2/
│   └── s3/
├── cloudformation/
│   └── contentsphere-stack.yaml
├── cdk/
└── documentation/
```

6. Terraform Modules

VPC Module: Creates the VPC, public subnet, Internet Gateway, public route table and route table association.

Security Group Module: Creates the security group with HTTP and SSH ingress and unrestricted outbound traffic.

EC2 Module: Selects the latest matching Ubuntu 24.04 AMI, launches a t3.micro instance and installs Apache using user data. An Elastic IP is attached to the instance.

S3 Module: Creates the ContentSphere application bucket using the AWS account ID to keep the bucket name unique.

7. Remote Terraform State

Terraform state is stored remotely in Amazon S3 instead of keeping the state only on the local management machine. The configured backend uses an S3 bucket named **contentsphere-terraform-state-888869353635** with the key **contentsphere/terraform.tfstate** in the ap-south-1 region. S3 versioning was enabled for the state bucket. This provides centralized state storage and protects the project from losing the local state file.

Remote backend configuration:

```
backend "s3" {
  bucket = "contentsphere-terraform-state-888869353635"
  key    = "contentsphere/terraform.tfstate"
  region = "ap-south-1"
  use_lockfile = true
}
```

8. EC2 Web Server Deployment

The EC2 module automatically selects an Ubuntu 24.04 AMI in ap-south-1. The instance type is t3.micro. User data installs Apache2, enables the service, starts it and writes a custom ContentSphere HTML page. The page contains the student name, batch code, AWS region and environment.

The final deployed Elastic IP was **15.252.229.172**. The web server returned HTTP 200 OK when tested with curl, confirming successful deployment and Apache availability.

9. Infrastructure Drift Detection

A drift test was performed by changing the S3 bucket Name tag outside Terraform. The modified tag was set to **DRIFT-TEST**. Terraform plan detected the difference and reported one resource requiring modification. Running terraform apply restored the tag to the desired value **contentsphere-bucket**.

This demonstrates one of the main advantages of Infrastructure as Code: the desired infrastructure configuration is stored as code and can be used to detect and correct configuration drift.

10. CloudFormation Implementation

A CloudFormation template was created at cloudformation/contentsphere-stack.yaml. The template was successfully validated using the AWS CloudFormation validation command, confirming that the template syntax was valid.

A deployment was also attempted. The stack could not be created because the AWS account had already reached the account limits for VPCs and Internet Gateways. CloudFormation reported maximum VPC and maximum Internet Gateway errors, after which the stack was removed. This was an AWS account service-limit issue rather than a template syntax problem.

11. AWS CDK Implementation

An AWS CDK project was created using TypeScript. The CDK stack defines a VPC, public subnet, security group, S3 bucket and EC2 instance. The EC2 instance uses a region-specific Ubuntu AMI and user data to install Apache and create the ContentSphere web page.

The CDK TypeScript project successfully completed npm run build and cdk synth. CDK deployment was not performed because deploying another VPC and Internet Gateway would encounter the same AWS account service limits described above.

12. Terraform vs CloudFormation vs AWS CDK

Feature	Terraform	CloudFormation	AWS CDK
Language	HCL	YAML	TypeScript
Provider	Multi-cloud / AWS	AWS	AWS
Project role	Primary implementation	Alternative implementation	Alternative implementation
Validation	terraform validate/plan	validate-template	npm build / cdk synth
Deployment	Successfully deployed	Blocked by AWS limits	Synthesized; deployment not performed
State	Terraform state in S3	AWS managed stack state	CloudFormation-backed

13. Validation and Testing

The final Terraform configuration was validated using terraform validate. Terraform plan was then executed against the real AWS environment. The final result was **No changes. Your infrastructure matches the configuration.**

Final Terraform outputs:

```
ec2_instance_id = "i-0d775dbe19d8588c3"
ec2_public_ip   = "15.252.229.172"
elastic_ip      = "15.252.229.172"
```

```
s3_bucket_name = "contentsphere-888869353635"
security_group_id = "sg-0be4d976a888669e9"
subnet_id = "subnet-09455376b70f5b2bc"
vpc_id = "vpc-02a0f7033138e2f5a"
```

Git verification also confirmed that the repository working tree was clean and synchronized with the main branch on GitHub. Terraform state files, plan files, .terraform directories and other generated artifacts were excluded using .gitignore.

14. Challenges Encountered

- AWS Internet Gateway and VPC account limits initially prevented some additional infrastructure from being created.
- An unused Internet Gateway had to be removed before the Terraform deployment could complete.
- The EC2 web server was replaced once to ensure correct initialization and website deployment.
- The installed AWS CDK version did not expose the expected latestUbuntu helper, so the implementation was adapted to use a region-specific generic Linux AMI.
- CloudFormation deployment was blocked by the same AWS account VPC/Internet Gateway service limits, although the template itself validated successfully.

15. Security Considerations

The security group was configured only for the services required by the demonstration: SSH on port 22 and HTTP on port 80. The project uses an example variables file rather than committing a terraform.tfvars file. Terraform state and generated plan files are excluded from Git.

For a production deployment, SSH should be restricted to a trusted source instead of 0.0.0.0/0, HTTPS should be used, IAM permissions should follow least privilege, and sensitive values should be stored in appropriate secret-management services.

16. Git and GitHub Version Control

The completed project was initialized as a Git repository and committed to the main branch. The final commit message was **Complete ContentSphere Terraform IaC project**. The repository was pushed successfully to GitHub and the local main branch was confirmed to be up to date with origin/main.

17. Learning Outcomes

- Understanding Infrastructure as Code and declarative infrastructure management.
- Creating reusable Terraform modules.
- Managing AWS networking resources through code.
- Automating EC2 server initialization with user data.
- Managing Terraform state remotely using Amazon S3.
- Detecting and correcting infrastructure drift.
- Understanding the differences between Terraform, CloudFormation and CDK.
- Using AWS CLI for validation and resource inspection.
- Using Git and GitHub for DevOps project version control.
- Troubleshooting AWS account limits and adapting infrastructure implementation accordingly.

18. Conclusion

The ContentSphere project successfully demonstrates an end-to-end AWS Infrastructure as Code workflow using Terraform. The required AWS infrastructure was provisioned, tested and brought into a state where terraform plan reports no differences. The EC2 web server was successfully deployed and verified, the S3 bucket was managed through Terraform, remote state was configured, and infrastructure drift was demonstrated and corrected.

CloudFormation and AWS CDK implementations were also prepared to demonstrate alternative IaC approaches. CloudFormation and CDK validation/synthesis succeeded, while deployment of those additional stacks was limited by AWS account service quotas. Overall, the project provides a practical demonstration of AWS DevOps, Infrastructure as Code, automation, version control and basic infrastructure lifecycle management.

End of Project Report